

Tarea-examen 3. Complejidad Computacional.

El Teorema de Cook-Levin: SAT y 3SAT son NP-completos

Indicaciones. Justifica todas las respuestas. Cuando demuestres una implicación, indica explícitamente la dirección que estás demostrando.

Los Problemas 3, 4 y 5 suman 10 puntos y son suficientes para obtener 10. Los Problemas 1, 2 y 6 son opcionales y sirven como puntos de recuperación: si alguno de los problemas principales no te sale completo, puedes compensar resolviendo alguno de estos. El total disponible es de **17 puntos**.

Entrega. El documento se entrega de forma **digital** a más tardar el **jueves 11 de junio**. Puedes escribir tus respuestas **a mano** (foto o escaneo legible) **o en L^AT_EX**. En cualquier caso, sube tu archivo a **Google Drive** o **GitHub** y comparte el enlace (con permisos de lectura) en la asignación de Classroom.

Problema 1 (2 puntos)

Consecuencia directa: $P = NP$ si y solo si $SAT \in P$.

En este problema formalizamos la razón por la que SAT es el problema “más difícil” de NP.

- a) (0.5 pts) ($\Rightarrow$) Supón que $P = NP$. Concluye que $SAT \in P$ y justifica brevemente por qué.
- b) (1 pt) ($\Leftarrow$) Supón ahora que $SAT \in P$. Sea $L \in NP$ un lenguaje arbitrario.
 - I. Explica por qué existe una reducción de Karp en tiempo polinomial f tal que $x \in L \iff f(x) \in SAT$. (*Basta citar el resultado correspondiente visto en clase.*)
 - II. Dado un algoritmo polinomial D para SAT, describe un algoritmo polinomial para L usando f y D .
 - III. Concluye que $L \in P$.
- c) (0.5 pts) Combina los incisos anteriores para enunciar la equivalencia $P = NP \iff SAT \in P$. ¿Por qué este resultado convierte a SAT en un problema “representante” de toda la clase NP?

Problema 2 (2 puntos)

Expresividad de fórmulas CNF.

Recordamos las siguientes definiciones. Un *literal* es una variable booleana x_i o su negación $\bar{x}_i$. Una *cláusula* es una disyunción de literales, es decir, una expresión de la forma $(\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$ donde cada ℓ_j es un literal. Una fórmula en *Forma Normal Conjuntiva* (CNF) es una conjunción de cláusulas:

$$\varphi = \bigwedge_{i=1}^m \left(\bigvee_j \ell_{ij} \right).$$

Antes de probar que SAT es NP-completo necesitamos saber “hablar en CNF”.

- a) (0.5 pts) La fórmula $(x_1 \vee \bar{y}_1) \wedge (\bar{x}_1 \vee y_1)$ está en forma CNF.
- I. Muestra que es satisfecha *si y solo si* $x_1 = y_1$.
 - II. Usando este hecho, escribe una fórmula CNF $\varphi_ =$ sobre las variables $x_1, \dots, x_n, y_1, \dots, y_n$ que es satisfecha si y solo si $x_i = y_i$ para todo $i \in \{1, \dots, n\}$. ¿Cuántas cláusulas tiene $\varphi_ =$?
- b) (1 pt) **Universalidad de AND/OR/NOT.** Sea $f : \{0, 1\}^\ell \rightarrow \{0, 1\}$ una función booleana cualquiera, con $\ell \in \mathbb{N}$ fijo.
- I. Para cada $v \in \{0, 1\}^\ell$ tal que $f(v) = 0$, construye una cláusula C_v en las ℓ variables tal que $C_v(v) = 0$ y $C_v(u) = 1$ para todo $u \neq v$.
Sugerencia: Si $v = (v_1, \dots, v_\ell)$, considera el literal $z_j = x_j$ si $v_j = 0$ y $z_j = \bar{x}_j$ si $v_j = 1$.
 - II. Define $\varphi_f = \bigwedge_{v: f(v)=0} C_v$. Demuestra que $\varphi_f(u) = f(u)$ para todo $u \in \{0, 1\}^\ell$, y acota el tamaño de φ_f .
- c) (0.5 pts) Sea $F : \{0, 1\}^k \rightarrow \{0, 1\}^c$ una función con k y c constantes (que dependen de la máquina, pero no del tamaño de la entrada).
- I. Explica por qué cada bit de salida de F puede expresarse como una fórmula CNF de tamaño *constante* (usando el inciso b).
 - II. ¿Por qué este hecho es crucial en la prueba del Teorema de Cook-Levin?

Problema 3 (5 puntos)

SAT $\leq_p$ 3SAT: la reducción cláusula por cláusula.

Recordamos que k -SAT es el lenguaje de las fórmulas CNF satisfacibles en las que cada cláusula tiene *a lo más* k literales. El objetivo de este problema es mostrar que toda fórmula CNF puede convertirse en una fórmula 3CNF equivalente (en cuanto a satisfacibilidad) en tiempo polinomial.

- a) (1 pt) **Cláusulas cortas.**
- I. Sea $C = (\ell_1)$ una cláusula de un solo literal. Introduce dos variables auxiliares z_1, z_2 y reemplaza C por una conjunción de cláusulas de exactamente 3 literales, preservando la satisfacibilidad.
 - II. Sea $C = (\ell_1 \vee \ell_2)$ una cláusula de dos literales. Introduce una variable auxiliar z y reemplaza C por una cláusula de exactamente 3 literales, preservando la satisfacibilidad.
- b) (1 pt) **Cláusula de 4 literales.** Sea $C = (\ell_1 \vee \ell_2 \vee \ell_3 \vee \ell_4)$. Introduce una variable auxiliar z y propón dos cláusulas C_1 y C_2 de exactamente 3 literales que reemplacen a C .
- c) (1.5 pts) **Correctud del reemplazo.** Demuestra que $C_1 \wedge C_2$ (del inciso b) es satisfacible si y solo si C lo es.

- I. ($\Rightarrow$) Si existe una asignación que satisface C , muestra que existe un valor para z que satisfice $C_1 \wedge C_2$.
 - II. ($\Leftarrow$) Si $C_1 \wedge C_2$ es satisfacible, muestra que la misma asignación (ignorando z) satisface C .
- d) (1 pt) **Generalización: cláusula de tamaño $k > 3$.** Sea $C = (\ell_1 \vee \ell_2 \vee \dots \vee \ell_k)$ con $k > 3$.
- I. Describe una construcción recursiva que reemplaza C por una conjunción de cláusulas de tamaño ≤ 3 , usando variables auxiliares $z_1, \dots, z_{k-3}$.
Sugerencia: En el paso base, aplica el inciso (b) con ℓ_1, ℓ_2 y z_1 para obtener una cláusula $(\ell_1 \vee \ell_2 \vee z_1)$ y luego recurre sobre $(\bar{z}_1 \vee \ell_3 \vee \dots \vee \ell_k)$.
 - II. ¿Cuántas variables auxiliares se introducen?
 - III. ¿Cuántas cláusulas de tamaño 3 produce la construcción?
 - IV. Argumenta (por inducción o por el mismo razonamiento del inciso c) que la conjunción resultante es satisfacible si y solo si C lo es.
- e) (0.5 pts) **La reducción f .** Sea φ una fórmula CNF con m cláusulas y longitud total $s = |\varphi|$ (número de símbolos). Aplica la construcción anterior cláusula por cláusula para obtener una fórmula 3CNF $\psi = f(\varphi)$.
- I. Acota el tamaño de ψ en función de s .
 - II. Muestra que f es computable en tiempo polinomial.
 - III. Concluye que $\text{SAT} \leq_p 3\text{SAT}$.

Problema 4 (2 puntos)

3SAT es NP-completo.

Usa los resultados de los problemas anteriores para concluir el Teorema de Cook-Levin.

- a) (0.5 pts) Muestra que $3\text{SAT} \in \mathbf{NP}$ describiendo un testigo y un verificador polinomial.
- b) (1 pt) Usa que SAT es NP-duro (visto en clase) y la reducción $\text{SAT} \leq_p 3\text{SAT}$ (Problema 3e) para mostrar que 3SAT es NP-duro. *Sugerencia:* usa la transitividad de las reducciones en tiempo polinomial.
- c) (0.5 pts) Concluye que 3SAT es NP-completo.

Problema 5 (3 puntos)

Autoreducibilidad de SAT: decidir implica encontrar.

En este problema mostramos que si $\mathbf{P} = \mathbf{NP}$ (y por lo tanto $\text{SAT} \in \mathbf{P}$), entonces no solo podemos *decidir* si una fórmula es satisfacible, sino también *encontrar* una asignación satisfactoria en tiempo polinomial.

Sea D un algoritmo en tiempo polinomial para SAT y sea φ una fórmula CNF satisfacible sobre las variables $x_1, \dots, x_n$. Para $b \in \{0, 1\}$, denotamos por $\varphi[x_i \leftarrow b]$ la fórmula obtenida al sustituir x_i por b y simplificar.

- a) (0.5 pts) Muestra que si $\varphi \in \text{SAT}$, entonces $\varphi[x_1 \leftarrow 0] \in \text{SAT}$ o $\varphi[x_1 \leftarrow 1] \in \text{SAT}$ (o ambas).
- b) (1 pt) Describe el siguiente algoritmo en pseudocódigo y argumenta que es correcto:
- I. Para $i = 1, \dots, n$: si $D(\varphi[x_i \leftarrow 0]) = 1$, fija $x_i = 0$; de lo contrario, fija $x_i = 1$. Actualiza φ sustituyendo el valor asignado a x_i .
 - II. Al terminar el ciclo, devuelve la asignación $(x_1, \dots, x_n)$.
- Muestra que, si $\varphi \in \text{SAT}$, el algoritmo devuelve una asignación que satisface φ .
- c) (0.5 pts) ¿Cuántas llamadas a D realiza el algoritmo? Concluye que el algoritmo corre en tiempo polinomial (suponiendo que D es polinomial).
- d) (1 pt) **Extensión a cualquier $L \in \text{NP}$.** Sea $L \in \text{NP}$ con reducción de Cook-Levin $x \mapsto \varphi_x$ (vista en clase), donde la asignación satisfactoria de φ_x incluye, entre sus variables, al testigo u de $x \in L$. Suponiendo $\text{P} = \text{NP}$, describe cómo encontrar un testigo para $x \in L$ en tiempo polinomial.

Problema 6 (3 puntos)

3SAT $\leq_p$ INDSET: las reducciones se propagan.

Definimos $\text{INDSET} = \{\langle G, k \rangle : G \text{ tiene un conjunto independiente de tamaño } k\}$. En este problema usamos que 3SAT es NP-completo (Problema 4) para mostrar que INDSET también lo es.

Sea $\varphi = C_1 \wedge \dots \wedge C_m$ una fórmula 3CNF con m cláusulas. Para cada cláusula $C_i = (\ell_{i,1} \vee \ell_{i,2} \vee \ell_{i,3})$ asociamos un *grupo* de tres vértices $\{v_{i,1}, v_{i,2}, v_{i,3}\}$ en una gráfica $G(\varphi)$. Las aristas de $G(\varphi)$ se agregan según las siguientes dos reglas:

- (R1) **Aristas internas.** Para cada grupo i y para todo par $j \neq j'$, agrega la arista $\{v_{i,j}, v_{i,j'}\}$.
- (R2) **Aristas de conflicto.** Para grupos distintos $i \neq i'$ y cualesquiera j, j' , agrega la arista $\{v_{i,j}, v_{i',j'}\}$ si $\ell_{i,j}$ y $\ell_{i',j'}$ son literales contradictorios (i.e. $\ell_{i,j} = x_k$ y $\ell_{i',j'} = \bar{x}_k$ para alguna variable x_k , o viceversa).

- a) (0.5 pts) Da un ejemplo pequeño: toma $\varphi = (x_1 \vee x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_2 \vee x_3)$, dibuja $G(\varphi)$ indicando vértices, aristas internas y aristas de conflicto, y señala un conjunto independiente de tamaño $m = 2$.
- b) (1 pt) (φ satisfacible $\Rightarrow G(\varphi)$ tiene un conjunto independiente de tamaño m .) Sea σ una asignación que satisface φ .
- I. Para cada cláusula C_i , elige un literal ℓ_{i,j_i} que sea verdadero bajo σ y sea el vértice representante de C_i . Define $S = \{v_{i,j_i} : 1 \leq i \leq m\}$.
 - II. Muestra que $|S| = m$.
 - III. Muestra que S es un conjunto independiente (usa las reglas R1 y R2 para descartar que haya aristas dentro de S).

- c) (1 pt) ($G(\varphi)$ tiene un conjunto independiente de tamaño $m \Rightarrow \varphi$ es satisfacible.) Sea S un conjunto independiente de tamaño m en $G(\varphi)$.
- I. Usa la regla R1 para mostrar que S contiene exactamente un vértice de cada grupo.
 - II. Usa la regla R2 para mostrar que los literales correspondientes a los vértices de S son pairwise consistentes (nunca aparecen x_k y $\bar{x}_k$ simultáneamente).
 - III. Construye una asignación σ que satisfaga φ .
- d) (0.5 pts) Muestra que la construcción $\varphi \mapsto \langle G(\varphi), m \rangle$ es computable en tiempo polinomial en $|\varphi|$. Concluye que $3\text{SAT} \leq_p \text{INDSET}$ y, usando que 3SAT es NP-duro y que $\text{INDSET} \in \mathbf{NP}$, que INDSET es NP-completo.